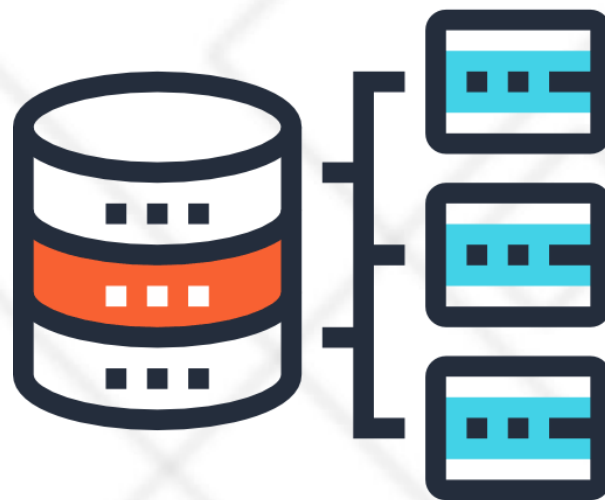




北京理工大学  
BEIJING INSTITUTE OF TECHNOLOGY



# 第4章 关系数据库 标准语言SQL

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn



# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

# 4.1 SQL简介

- 结构化查询语言SQL (Structured Query Language) 是一种介于关系代数与关系演算之间的语言，是一个通用的、功能极强的关系数据库语言，是关系数据库的标准语言
- SQL语言的版本包括：
  - SQL-89, SQL-92
  - SQL-99 (SQL3) 增加了面向对象的概念
  - SQL2003 (SQL4)
- SQL语言集数据查询、数据操纵、数据定义和数据控制功能于一体，充分体现了关系数据语言的特点和优点

## □ SQL示例

```
SELECT Sno, Sname  
FROM Student  
WHERE Sdept='计算机'  
ORDER BY Sno;
```

- 从Student表中，选取：学号Sno和姓名Sname，满足条件：所在系Sdept='计算机'，按照学号升序排列

## □综合统一

- 集数据定义语言DDL、数据操纵语言DML、数据控制语言DCL的功能于一体，可以完成数据库生命周期中的全部活动
- 关系模型中实体和实体间的联系都用关系来表示，使得操作符单一，每种操作只使用一个操作符

## □高度非过程化

- 使用SQL语言，只需要提出“做什么”，而无需指明“怎么做”，无需了解存取路径，提高了数据独立性

## □面向集合的操作方式

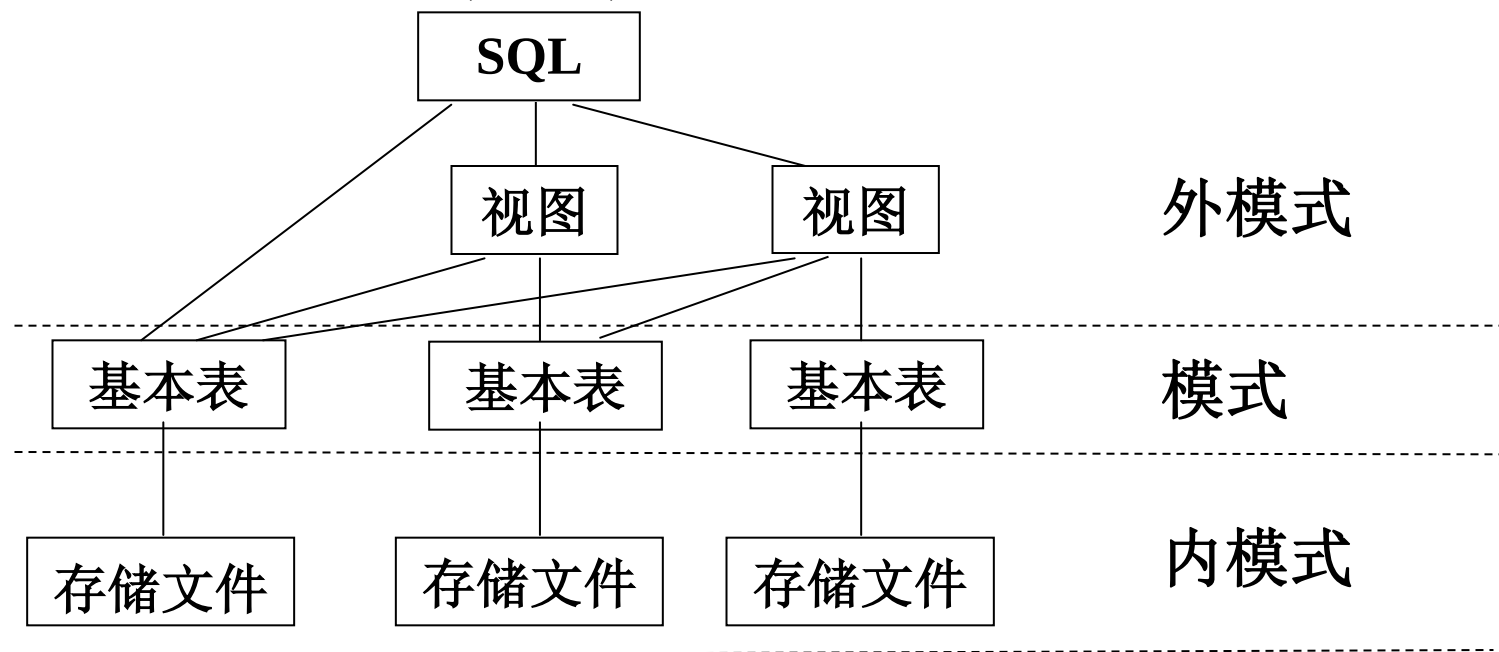
- SQL语言采用集合操作方式，查询、插入、删除、修改操作的对象都是集合

- 以同一种语法结构提供多种使用方式
  - 作为独立的语言
    - ◆ 提供联机交互工具，在终端键盘上直接键入SQL命令对数据库进行操作，由DBMS来进行解释
  - 作为嵌入式语言
    - ◆ SQL语句能嵌入到高级语言程序中，使应用程序充分利用SQL访问数据库的能力
- 不同方式下，SQL的语法结构基本上是一致的，提供了极大的灵活性和方便性
- 语言简洁，易学易用

## 4.2 SQL的系统结构

### □ SQL语言支持数据库的三级模式结构

- 在SQL中，关系对应于基本表 (Table)，基本表的集合形成数据库模式，对应三级模式结构的模式
- 基本表在物理上与存储文件相对应，所有存储文件的集合为物理数据库
- 外模式由视图 (View) 组成



## 4.2 SQL的系统结构

### □基本表和视图的关系

- 基本表 (Base Table, Table)

- ◆ 独立存在的表

- ◆ 一个关系对应一个基本表

- 视图 (View)

- ◆ 是从一个或多个基本表中导出的表，仅有逻辑上的定义，不实际存储数据，是一种**虚表**

- ◆ 视图的定义存储在数据字典中，在使用的时候，根据定义从基本表中导出数据供用户使用

- ◆ 视图可以像基本表一样进行查询和**某些更新**操作

### □两者都可称为SQL的表，但是有区别





# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

## 4.3 数据定义

- SQL的数据定义功能主要包括定义表、定义视图和定义索引，在SQL2中还增加了对SQL数据库模式的定义

表 SQL 的数据定义语句

| 操 作<br>对 象 | 操 作 方 式              |                    |                    |
|------------|----------------------|--------------------|--------------------|
|            | 创 建                  | 删 除                | 修 改                |
| 模式         | <b>CREATE SCHEMA</b> | <b>DROP SCHEMA</b> |                    |
| 表          | <b>CREATE TABLE</b>  | <b>DROP TABLE</b>  | <b>ALTER TABLE</b> |
| 视 图        | <b>CREATE VIEW</b>   | <b>DROP VIEW</b>   |                    |
| 索 引        | <b>CREATE INDEX</b>  | <b>DROP INDEX</b>  |                    |

## 4.3.1 模式的定义和删除

- 在SQL中，一个SQL模式 (SQL Schema) 由模式名、权限标识符和模式中元素的描述符组成
  - 权限标识符指明拥有该模式的用户或帐号
  - 模式元素包含一个数据库应用的表、视图和索引等
  - 属于同一应用的表、视图和索引等可以定义在同一模式中
- 在定义模式时可先给出模式名和权限标识符，以后再定义其中的元素，语法格式：

CREATE SCHEMA <模式名> AUTHORIZATION <用户名>;

- 如果没有指定<模式名>，则隐含为<用户名>

## 4.3.1 模式的定义和删除

□[例4-1] 定义学生数据库模式SST，用户为SDBA

```
CREATE SCHEMA SST  
AUTHORIZATION SDBA;
```

□[例] CREATE SCHEMA

```
AUTHORIZATION WANG;
```

- 没有指定“模式名”，所以“模式名”隐含为用户名WANG

## 4.3.1 模式的定义和删除

- 定义模式后，实际上定义了一个命名空间，可以进一步定义该模式包含的数据库对象，如表，视图和索引等
- 可以在创建模式的同时在模式定义中进一步创建基本表、视图、定义授权等

CREATE SCHEMA <模式名> AUTHORIZATION <用户名>  
[<表定义子句> | <视图定义子句> | <授权定义子句>];

- 例：创建相互依赖的 FOREIGN KEY 约束

```
CREATE SCHEMA AUTHORIZATION ross
    CREATE TABLE t1 (c1 INT PRIMARY KEY,
                     c2 INT REFERENCES t2(c1))
    CREATE TABLE t2 (c1 INT PRIMARY KEY,
                     c2 INT REFERENCES t1(c1));
```

## 4.3.1 模式的定义和删除

### □ 删除模式语句：

**DROP SCHEMA <模式名> [CASCADE | RESTRICT];**

- CASCADE（级联式）方式

- ◆ 表示在删除模式的同时把该模式中所有的数据库对象全部一起删除

- RESTRICT（限制式）方式

- ◆ 表示如果该模式中已经定义了下属的数据库对象，则拒绝该删除语句的执行

## 4.3.2 表的定义、删除与修改

### 1. 定义基本表

CREATE TABLE <表名>

(<列名> <数据类型> [<列级完整性约束条件>]

[, <列名> <数据类型> [<列级完整性约束条件>]] ...

[, <表级完整性约束条件>]);

- <表名>：所要定义的基本表的名称
- <列名>：组成该表的各个属性（列）
- <列级完整性约束条件>：涉及相应属性列的完整性约束条件
- <表级完整性约束条件>：涉及一个或多个属性列的完整性约束条件

# 1. 定义基本表

□[例4-3] 建立学生表Student，表中属性有：学号Sno，姓名Sname，年龄Sage，性别Ssex，学生所在系Sdept

```
CREATE TABLE Student
(Sno CHAR(6) NOT NULL UNIQUE,
Sname CHAR(8),
Sage INT,
Ssex CHAR(2),
Sdept CHAR(12),
CONSTRAINT C1 CHECK (Ssex IN ('男','女')),
CONSTRAINT S_PK PRIMARY KEY (Sno));
```

- CONSTRAINT子句定义列级或表级约束，其格式为  
CONSTRAINT <约束名> <约束>



# 1. 定义基本表

- 在SQL2中增加了定义域的语句，可以用域名代替指定列的数据类型
- 如果有一个或多个表的属性的域是相同的，通过对域的修改可以很容易地改变属性的数据类型
- 域定义语句的格式为：

`CREATE DOMAIN <域名> <数据类型>;`

[例]

`CREATE DOMAIN Sdept_TYPE CHAR(12);`

- 域Sdept\_TYPE创建后，定义学生表时，对列Sdept的类型定义可以用域名代替：`Sdept Sdept_TYPE`

# 1. 定义基本表

## [例4-4]

```
CREATE TABLE Course
  (Cno CHAR(6) NOT NULL,
   Cname CHAR(20),
   Ccredit INT,
   CONSTRAINT C_PK PRIMARY KEY (Cno));
```

```
CREATE TABLE SC
  (Sno CHAR(6) NOT NULL,
   Cno CHAR(6) NOT NULL,
   Grade INT CHECK (Grade BETWEEN 0 AND 100),
   CONSTRAINT SC_PK PRIMARY KEY(Sno,Cno),
   CONSTRAINT SC_FK1 FOREIGN KEY (Sno)
REFERENCES Student(Sno),
   CONSTRAINT SC_FK2 FOREIGN KEY (Cno)
REFERENCES Course(Cno));
```

# 1. 定义基本表

## [例4-4]

CREATE TABLE Course

(Cno CHAR(6) NOT NULL,

Cname CHAR(20),

Ccredit INT,

PRIMARY KEY (Cno));

CREATE TABLE SC

(Sno CHAR(6) NOT NULL,

Cno CHAR(6) NOT NULL,

Grade INT CHECK (Grade BETWEEN 0 AND 100),

PRIMARY KEY (Sno,Cno),

FOREIGN KEY (Sno) REFERENCES Student(Sno),

FOREIGN KEY (Cno) REFERENCES Course(Cno));

# 1. 定义基本表

□在定义基本表时，表所属的数据库模式一般被隐式指定，也可以显式地在定义表时指定表所属的数据库模式名

- 如下语句在定义学生表时，同时指出学生表所在的模式为学生数据库模式SST

```
CREATE TABLE SST.Student  
    (Sno CHAR(6) NOT NULL UNIQUE, ...);
```

□在创建模式语句中同时创建表

## 2. 修改基本表

ALTER TABLE <表名>

[ **ADD** <列名> <数据类型> [<完整性约束>] ]

[ **DROP** <列名> [CASCADE | RESTRICT] ]

[ **ALTER** <列名> <数据类型> ];

- **ADD子句**用于增加新列，包括列名、数据类型和列级完整性约束
- **DROP子句**用于删除指定的列名
  - ◆ CASCADE表示删除列时自动删除引用该列的视图和约束
  - ◆ RESTRICT表示没有视图和约束引用时才能删除该列，否则将拒绝删除操作
- **ALTER子句**用于修改列的定义，如修改列的数据类型或修改列的宽度

## 2. 修改基本表

□[例4-5] 在学生表Student中增加一列，列名为班级

```
ALTER TABLE Student
```

```
ADD Class CHAR(8);
```

- 不论基本表中原来是否已有数据，新增加的列一律为空值

□[例4-6] 修改学生表Student中姓名列的长度为20

```
ALTER TABLE Student
```

```
ALTER Sname CHAR(20);
```

- 修改原有的列定义有可能会破坏已有数据

## 3. 删除基本表

**DROP TABLE <表名> [RESTRICT | CASCADE];**

- 若选择 RESTRICT，则删除的基本表不能是由 FOREIGN KEY 约束引用的表，不能有视图，不能有触发器，不能有存储过程或函数等。如果存在这些依赖该表的对象，则此表不能被删除
- 若选择 CASCADE，则该表的删除没有限制条件。在删除基本表的同时，相关的依赖对象，例如视图等都将一起删除
- 在缺省的情况下，默认为 RESTRICT

## 4.3.3 索引的建立与删除

- 索引是一种数据结构，可以提高查询速度。大型关系数据库中的索引的实现技术是数据库管理系统实现中重要的核心问题
- 当关系变得很大时，通过扫描所有关系中所有元组来找出那些（可能数量很少）匹配给定条件的元组的操作方式代价太高。例如

```
SELECT * FROM Movie WHERE studioName='Disney'  
AND year = 1990
```

- 方法1：取得全部元组，用WHERE子句逐一匹配
- 方法2：取得1990年的元组，判断studioName='Disney'。在year上创建索引，SQL查询处理器进行年份查询时，仅对指定年份的元组测试



## 4.3.3 索引的建立与删除

### □ 建立索引是加快查询速度的有效手段

- 可根据需要在在一个表上建立一个或多个索引，以提供多种存取路径，加快查找速度。也可不建

### □ 建立索引

- DBA或表的属主（即建立表的人）根据需要建立
- 有些DBMS自动建立以下列上的索引
  - ◆ PRIMARY KEY, UNIQUE

### □ 维护索引：DBMS自动完成

### □ 使用索引

- 系统在存取数据时会自动选择是否使用索引或以合适的索引作为存取路径，用户不必也不能选择索引

# 1. 建立索引

## □ 语句格式

**CREATE [UNIQUE] [CLUSTER] INDEX <索引名>  
ON <表名> (<列名> [<次序>] [, <列名> [<次序>]]...);**

- <表名>指定要建索引的基本表名字
- 索引可以建立在该表的一列或多列上，各列名之间用逗号分隔
- <次序>指定索引值的排列次序，升序ASC，降序DESC。缺省值：ASC
- UNIQUE表明此索引的每一个索引值只对应唯一的数据记录
- CLUSTER表示要建立的索引是聚簇索引

# 1. 建立索引

□[例4-8] 在学生表Student的列学号上按升序建立唯一索引

```
CREATE UNIQUE INDEX S_SNO  
ON Student(Sno);
```

- 对于已含重复值的属性列不能建UNIQUE索引
- 对某个列建立UNIQUE索引后，插入新记录时DBMS会自动检查新记录在该列上是否取了重复值

□[例4-9] 在表Student上，班级按降序、年龄按升序建立索引

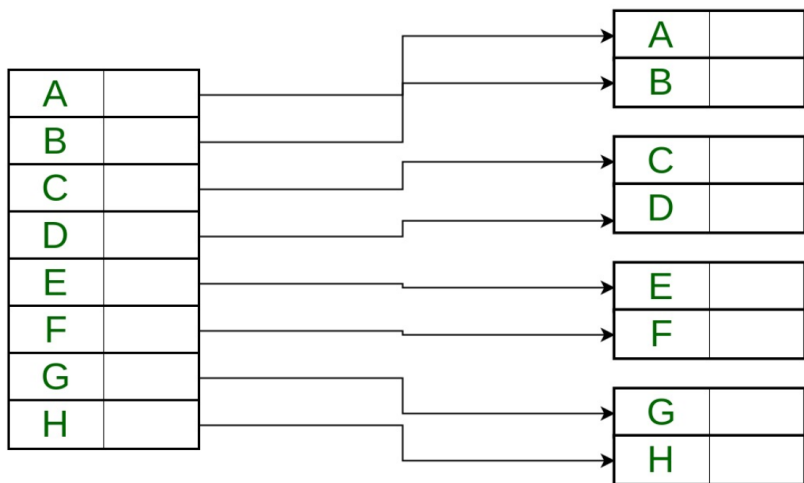
```
CREATE INDEX SCLASS_AGE  
ON Student (CLASS DESC, Sage ASC);
```

- 当班级值相同时将按年龄升序排列

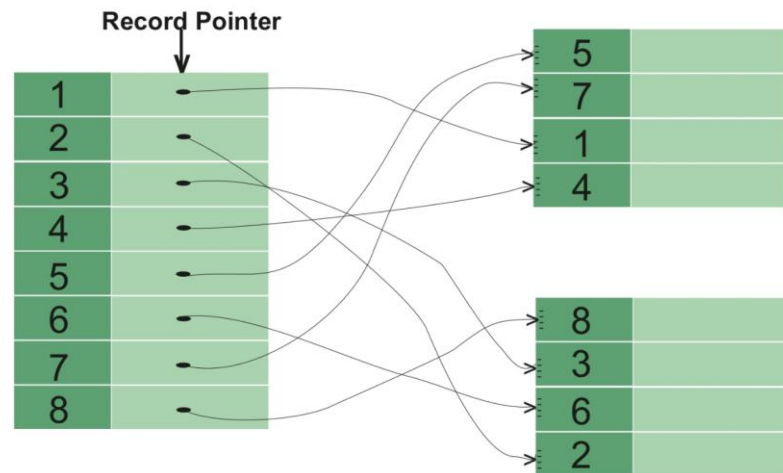
# 1. 建立索引

## □ 聚簇索引 (Cluster Index)

- 索引次序与基本表中元组的物理次序一致的索引
- 建立聚簇索引后，基本表中数据也需要按指定的聚簇属性值的升序或降序存放，即：聚簇索引的索引项顺序与表中记录的物理顺序一



聚簇索引



非聚簇索引

# 1. 建立索引

## □ 聚簇索引 (Cluster Index)

- 索引次序与基本表中元组的物理次序一致的索引
- 建立聚簇索引后，基本表中数据也需要按指定的聚簇属性值的升序或降序存放，即：聚簇索引的索引项顺序与表中记录的物理顺序一

## □ [例]

```
CREATE CLUSTER INDEX S_Sname  
ON Student(Sname);
```

- 在Student表的Sname（姓名）列上建立一个聚簇索引，而且Student表中的记录将按照Sname值的升序存放

# 1. 建立索引

## □ 聚簇索引确定表中数据的物理顺序

- 一个表只能包含一个聚簇索引。但该索引可以包含多个列（组合索引）
- 聚簇索引适用于：很少对基本表进行增删操作；很少对其中的变长列进行修改操作

## □ 聚簇索引对于那些经常要搜索范围值的列特别有效

- 使用聚簇索引找到包含第一个值的行后，便可以确保包含后续索引值的行是物理相邻的。有助于提高搜索范围值类查询的性能
- 如果对从表中检索的数据进行排序时经常要用到某一列，则可以将该表在该列上聚簇（物理排序），避免每次查询该列时都进行排序

## 2. 删除索引

### □ DROP INDEX <索引名>;

- 删除索引时，系统会从数据字典中删去有关该索引的描述

### □ [例4-10] 删除学生表上建立的S\_SNO索引

DROP INDEX S\_SNO;



# 索引选择



- 索引为性能所带来的好处却是有代价的。数据库设计者需要对索引作一个折中的选择，这种选择是衡量数据库设计成败的重要因素
  - 对某个属性使用索引能极大提高对该属性上的值的检索效率，使用到该属性时，还可以加快连接
  - 带索引的表在数据库中会占据更多的空间
  - 维护索引的代价，对数据进行插入、更新、删除操作的命令所花费的时间会更长
- 应评估两个因素：数据的性质和基于表的查询的性质，来决定是否创建索引，应确保对性能的提高程度大于在存储空间和处理资源方面的代价





# 第4章 关系数据库标准语言SQL



- 4.1 SQL简介
- 4.2 SQL的系统结构
- 4.3 SQL的数据定义
- 4.4 SQL的数据操纵
- 4.5 SQL中的视图
- 4.6 SQL的数据控制
- 4.7 嵌入式SQL
- 4.8 小结

## 4.4 SQL的数据操纵

SQL的数据操纵包括数据查询、插入、修改和删除

### □ 4.4.1 数据查询

- 1. 单表查询
- 2. 连接查询
- 3. 嵌套查询
- 4. 集合查询

### □ 4.4.2 数据更新

## 4.4.1 查询

### □ 语句格式

**SELECT** [ ALL|DISTINCT ] <目标列表达式>[, <目标列表达式>] ...

**FROM** <表名或视图名>[, <表名或视图名>] ...

[ **WHERE** <条件表达式> ]

[ **GROUP BY** <列名1> [ **HAVING** <条件表达式> ] ]

[ **ORDER BY** <列名2> [ ASC|DESC ] ];

- **SELECT子句**：指定要显示的属性列
- **FROM子句**：指定查询对象（基本表或视图）
- **WHERE子句**：指定查询条件
- **GROUP BY子句**：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用聚集函数
- **HAVING短语**：筛选出只有满足指定条件的组
- **ORDER BY子句**：对查询结果表按指定列值的升序或降序排序

## 4.4.1 查询

**SELECT** A1, A2, ..., An

**FROM** R1, R2, ..., Rm

**WHERE** P

$$\Pi_{A1, A2, \dots, An}(\sigma_p(R_1 \times R_2 \times \dots \times R_m))$$

SELECT

WHERE

FROM

- 关系代数中基于关系是一个集合这样的数学概念，因此重复的元组不会在关系中出现
- 但在实践中，要删除查询结果中的重复元组是相当费时的，所以在商用数据库产品中，允许在关系和SQL表达式的结果中出现重复元组

# 示例：学生-课程数据库

## □学生表：

Student(Sno, Sname, Sage, Ssex, Sdept)

学生表(学号, 姓名, 年龄, 性别, 所在系)

## □课程表：

Course(Cno, Cname, Cpno, Ccredit)

课程表(课程号, 课程名, 先行课, 学分)

## □学生选课表：

SC(Sno, Cno, Grade)

学生选课表(学号, 课程号, 成绩)

# 示例：学生-课程数据库



| Sno   | Sname | Ssex | Sage | Sdept |
|-------|-------|------|------|-------|
| 95001 | 李勇    | 男    | 20   | CS    |
| 95002 | 刘晨    | 女    | 19   | CS    |
| 95003 | 王敏    | 女    | 18   | MA    |
| 95004 | 张立    | 男    | 19   | IS    |

**Student**

| Sno   | Cno | Grade |
|-------|-----|-------|
| 95001 | 1   | 92    |
| 95001 | 2   | 85    |
| 95001 | 3   | 88    |
| 95002 | 2   | 90    |
| 95002 | 3   | 80    |

**SC**

**Course**

| Cno | Cname    | Cpno | Ccredit |
|-----|----------|------|---------|
| 1   | 数据库      | 5    | 4       |
| 2   | 数学       |      | 2       |
| 3   | 信息系统     | 1    | 4       |
| 4   | 操作系统     | 6    | 3       |
| 5   | 数据结构     | 7    | 4       |
| 6   | 数据处理     |      | 2       |
| 7   | PASCAL语言 | 6    | 4       |

# 1. 单表查询

□ [例4-11] 查询CS系学生的学号和姓名

```
SELECT Sno, Sname FROM Student  
WHERE Sdept='CS';
```

□ [例4-12] 查询选修了课的学生学号，并按学号升序排列

```
SELECT DISTINCT Sno FROM SC  
ORDER BY Sno;
```

□ [例4-13] 查询年龄在18到25岁之间的学生信息

```
SELECT * FROM Student  
WHERE Sage BETWEEN 18 AND 25;
```

# 1. 单表查询

## □ 查询经过计算的值

- SELECT子句的<目标列表达式>不仅可以是表中的属性列，也可以是表达式
  - ◆ 算术表达式、字符串常量、函数、列别名等

## □ [例4-14] 查询全体学生的姓名及其出生年份

**SELECT Sname, 2025-Sage FROM Student;**

输出结果：

| Sname | 2025-Sage |
|-------|-----------|
| ----- | -----     |
| 李勇    | 2005      |
| 刘晨    | 2006      |
| 王敏    | 2007      |
| 张立    | 2006      |



# 1. 单表查询

- 查询全体学生的姓名、出生年份和所在系，要求用小写字母表示所在系名

```
SELECT Sname, 'Year of Birth:', 2025-Sage,  
       LOWER(Sdept) FROM Student;
```

输出结果：

| Sname | 'Year of Birth:' | 2025-Sage | LOWER(Sdept) |
|-------|------------------|-----------|--------------|
| 李勇    | Year of Birth:   | 2005      | cs           |
| 刘晨    | Year of Birth:   | 2006      | cs           |
| 王敏    | Year of Birth:   | 2007      | ma           |
| 张立    | Year of Birth:   | 2006      | is           |

# 1. 单表查询

□使用列别名改变查询结果的列标题:

```
SELECT Sname AS NAME, 'Year of Birth:' BIRTH,  
2025-Sage BIRTHDAY, LOWER(Sdept) DEPARTMENT  
FROM Student;
```

输出结果:

| NAME | BIRTH          | BIRTHDAY | DEPARTMENT |
|------|----------------|----------|------------|
| 李勇   | Year of Birth: | 2005     | cs         |
| 刘晨   | Year of Birth: | 2006     | cs         |
| 王敏   | Year of Birth: | 2007     | ma         |
| 张立   | Year of Birth: | 2006     | is         |

# 1. 单表查询

## □ 消除取值重复的行

- 在SELECT子句中使用DISTINCT短语

假设SC表中有下列数据

| Sno   | Cno   | Grade |
|-------|-------|-------|
| ----- | ----- | ----- |
| 95001 | 1     | 92    |
| 95001 | 2     | 85    |
| 95001 | 3     | 88    |
| 95002 | 2     | 90    |
| 95002 | 3     | 80    |

(2) **SELECT DISTINCT Sno  
FROM SC;**

结果: Sno

-----  
95001  
95002

**[例] 查询选修了课程的学生学号**

(1) **SELECT Sno FROM SC;**

**或 (默认 ALL)**

**SELECT ALL Sno FROM SC;**

结果: Sno

-----  
95001  
95001  
95001  
95002  
95002

# 1. 单表查询

□ 注意DISTINCT短语的作用范围是所有目标列

□ 例：查询选修课程的各种成绩

- 错误的写法

```
SELECT DISTINCT Cno, DISTINCT Grade  
FROM SC;
```

- 正确的写法

```
SELECT DISTINCT Cno, Grade  
FROM SC;
```

# 1. 单表查询

## □ WHERE子句常用的查询条件

|         |                                                         |
|---------|---------------------------------------------------------|
| 比较表达式   | <列名1> 比较算符 <列名2（或常量）><br>比较算符：=、>、>=、<、<=、<>（或!=）       |
| 逻辑表达式   | <条件表达式1> 逻辑算符 <条件表达式2><br>逻辑算符：AND、OR、NOT               |
| BETWEEN | <列名1> [NOT] BETWEEN <常量1或列名2> AND<br><常量2或列名3>          |
| IN      | <列名> [NOT] IN（常量表列 或 SELECT语句）                          |
| LIKE    | <列名> [NOT] LIKE ‘匹配字符串’<br>匹配符：“_”表示匹配一个字符，“%”表示匹配任意字符串 |
| NULL    | <列名> IS [NOT] NULL                                      |
| EXISTS  | [NOT] EXISTS（SELECT语句）                                  |

# (1) 比较大小

□ 在WHERE子句的 <比较条件> 中使用比较运算符

- =, >, <, >=, <=, != 或 <>, !>, !<
- 逻辑运算符NOT + 比较运算符

□ **[例]** 查询所有年龄在20岁以下的学生姓名及其年龄

```
SELECT Sname, Sage FROM Student  
WHERE Sage<20;
```

或

```
SELECT Sname, Sage FROM Student  
WHERE NOT Sage>=20;
```

## (2) 确定范围

□使用谓词 BETWEEN ... AND ...

NOT BETWEEN ... AND ...

□[例] 查询年龄在20~23岁（包括20岁和23岁）之间的学生的姓名、系别和年龄

```
SELECT Sname, Sdept, Sage FROM Student  
WHERE Sage BETWEEN 20 AND 23;  
(WHERE Sage>=20 AND Sage<=23)
```

□[例] 查询年龄不在20~23岁之间的学生姓名、系别和年龄

```
SELECT Sname, Sdept, Sage FROM Student  
WHERE Sage NOT BETWEEN 20 AND 23;
```

## (3) 确定集合

□使用谓词 IN <值表>, NOT IN <值表>

- <值表>: 用逗号分隔的一组取值

□[例] 查询信息系 (IS)、数学系 (MA) 和计算机科学系 (CS) 学生的姓名和性别

```
SELECT Sname, Ssex FROM Student
```

```
WHERE Sdept IN ('IS', 'MA', 'CS');
```

```
(WHERE Sdept='IS' OR Sdept='MA' OR Sdept='CS')
```

□[例] 查询既不是信息系、数学系, 也不是计算机科学系的学生姓名和性别

```
SELECT Sname, Ssex FROM Student
```

```
WHERE Sdept NOT IN ('IS', 'MA', 'CS');
```



## (4) 字符串匹配

[NOT] LIKE ‘<匹配串>’ [ESCAPE ‘<换码字符>’]

□<匹配串>：指定匹配模板

- 匹配模板：固定字符串或含通配符的字符串
  - ◆ 当匹配模板为固定字符串时，可以用 = 运算符取代 LIKE 谓词，用 != 或 <> 运算符取代 NOT LIKE 谓词
- 通配符
  - ◆ % 代表任意长度（长度可以为0）的字符串  
例：a%b表示以a开头，以b结尾的任意长度的字符串。如 acb, addgb, ab等都满足该匹配串
  - ◆ \_(下横线) 代表任意单个字符  
例：a\_b表示以a开头，以b结尾的长度为3的任意字符串。如acb, afb等都满足该匹配串

## (4) 字符串匹配

[NOT] LIKE ‘<匹配串>’ [ESCAPE ‘<换码字符>’]

□ESCAPE 短语：

- 当用户要查询的字符串本身就含有 % 或 \_ 时，要使用 ESCAPE ‘<换码字符>’ 短语对通配符进行转义

## (4) 字符串匹配

□ 匹配模板为含通配符的字符串

□[例] 查询所有姓刘学生的姓名、学号和性别

```
SELECT Sname, Sno, Ssex
```

```
FROM Student
```

```
WHERE Sname LIKE '刘%';
```

□[例] 查询姓“欧阳”且全名为三个汉字的学生的姓名

```
SELECT Sname
```

```
FROM Student
```

```
WHERE Sname LIKE '欧阳_ _';
```

## (4) 字符串匹配

□ 使用换码字符将通配符转义为普通字符

□ **[例]** 查询“DB\_Design”课程的课程号和学分

```
SELECT Cno, Ccredit
```

```
FROM Course
```

```
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```

□ **[例]** 查询以“DB\_”开头，且倒数第3个字符为i的课程的具体情况

```
SELECT * FROM Course
```

```
WHERE Cname LIKE 'DB\_%i\__' ESCAPE '\';
```

## (5) 涉及空值的查询

- 使用谓词 IS NULL 或 IS NOT NULL
- “IS NULL” 不能用 “= NULL” 代替
- **[例]** 某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号

**SELECT Sno, Cno FROM SC**

**WHERE Grade IS NULL;**

- **[例]** 查所有有成绩的学生学号和课程号

**SELECT Sno, Cno FROM SC**

**WHERE Grade IS NOT NULL;**

## □ SQL允许属性有一个特殊值NULL称作空值

- 未知值：有值但是不知道是什么，例如未知生日
- 不适用的值：例如配偶的名字
- 保留的值：无权知道的值，例未公布的电话号码

## □ 空值的运算

- 空值不同于空白或零值。没有两个相等的空值。空值和任何值进行算术运算，结果仍为空值
  - ◆ 执行计算时消除空值很重要，因为包含空值列的某些计算（如平均值）会不准确
- 当使用逻辑运算符和比较运算符，有可能返回结果UNKNOWN，是与TRUE和FALSE相同的布尔值

## □空串指的是零长度字符串

- 当m为0或负数时, RIGHT('123', m) 返回空字符串, RTRIM(' ') 返回空字符串

## (6) 多重条件查询

- 用逻辑运算符AND和OR来联结多个查询条件
  - AND的优先级高于OR
  - 可以用括号改变优先级
- 可用来实现多种其他谓词
  - [NOT] IN
  - [NOT] BETWEEN ... AND ...
- **[例]** 查询计算机系年龄在20岁以下的学生姓名

**SELECT Sname**

**FROM Student**

**WHERE Sdept='CS' AND Sage<20;**



# 对查询结果排序

## □使用ORDER BY子句

- 可以按一个或多个属性列排序
- 升序：ASC；降序：DESC；缺省值为升序

## □空值将作为最大值排序

- ASC：排序列为空值的元组最后显示
- DESC：排序列为空值的元组最先显示

□[例] 查询选修了3号课程的学生们的学号及其成绩，查询结果按分数降序排列

```
SELECT Sno, Grade  
FROM SC  
WHERE Cno='3'  
ORDER BY Grade DESC;
```

# 使用聚集函数

□ SQL提供了多种聚集函数，用来实现统计查询

- 计数

- ◆ COUNT([DISTINCT|ALL] \*)

- ◆ COUNT([DISTINCT|ALL] <列名>)

- 计算总和     SUM([DISTINCT|ALL] <列名>)

- 计算平均值   AVG([DISTINCT|ALL] <列名>)

- 求最大值     MAX([DISTINCT|ALL] <列名>)

- 求最小值     MIN([DISTINCT|ALL] <列名>)

□ DISTINCT短语：在计算时要取消指定列中的重复值

□ ALL短语：不取消重复值；**ALL**为缺省值

# 使用聚集函数

□[例] 查询学生总人数

```
SELECT COUNT(*) FROM Student;
```

□[例] 查询选修了课程的学生人数

```
SELECT COUNT(DISTINCT Sno) FROM SC;
```

注：用DISTINCT以避免重复计算学生人数

□[例] 计算1号课程的学生平均成绩

```
SELECT AVG(Grade) FROM SC
```

```
WHERE Cno='1';
```

□[例] 查询选修1号课程的学生最高分数

```
SELECT MAX(Grade) FROM SC
```

```
WHERE Cno='1';
```

# 对查询结果分组

## 使用GROUP BY子句分组

### □细化聚集函数的作用对象

- 未对查询结果分组，聚集函数将作用于整个查询结果
- 对查询结果分组后，聚集函数将分别作用于每个组

### □分组方法：按指定的一列或多列值分组，值相等的为一组

- 使用GROUP BY子句后，SELECT子句的列名列表中只能出现分组属性和聚集函数

### □[例] 求各个课程号及相应的选课人数

```
SELECT Cno, COUNT(Sno) FROM SC  
GROUP BY Cno;
```

# 对查询结果分组

- GROUP BY子句的作用对象是查询的中间结果表
- 使用HAVING短语筛选最终输出结果
  - 只有满足HAVING短语指定条件的组才输出
- **[例]** 查询选修了3门以上课程的学生学号

**SELECT Sno**

**FROM SC**

**GROUP BY Sno**

**HAVING COUNT(\*)>3;**

# 对查询结果分组

## □ HAVING短语与WHERE子句的区别

- 作用对象不同
  - ◆ WHERE子句作用于基表或视图，从中选择满足条件的元组 (tuple)
  - ◆ HAVING短语作用于组，从中选择满足条件的组 (group)
- WHERE子句中不能使用聚集函数
- HAVING短语中可以使用聚集函数

## 4.4.1 数据查询

### □1. 单表查询

### □2. 连接查询

- 连接查询是关系数据库中最主要的查询
- 等值连接、自然连接、非等值连接、自身连接、外连接、复合条件连接

### □3. 嵌套查询

### □4. 集合查询



## 2. 连接查询

- 同时涉及多个表的查询称为连接查询
  - 用来连接两个表的条件称为连接条件或连接谓词
  - 连接谓词中的列名称为连接字段
    - ◆ 连接条件中的各连接字段类型必须是可比的，但不必是相同的
- SQL中连接查询的主要类型
  - (1) 广义笛卡尔积
  - (2) 等值（含自然连接）与非等值连接查询
  - (3) 自身连接查询
  - (4) 外连接查询
  - (5) 复合条件连接查询

# (1) 广义笛卡尔积

- 不带连接谓词的连接（即没有WHERE子句）
- 广义笛卡尔积是两表元组的交叉乘积，其连接的结果会产生没有意义的元组，实际上很少使用

**[例]**

```
SELECT Student.*, SC.*  
FROM Student, SC;
```

## (2) 等值与非等值连接查询

□等值连接、自然连接，和非等值连接

□等值连接

- 连接运算符为 = 的连接操作

**[<表名1>.]<列名1> = [<表名2>.]<列名2>**

- 任何子句中引用表1和表2中的同名属性时，都必须加表名前缀
- 引用唯一属性名时可以加也可以省略表名前缀

□[例] 查询每个学生及其选修课程的情况

```
SELECT Student.*, SC.*
```

```
FROM Student, SC
```

```
WHERE Student.Sno = SC.Sno;
```

**Student**

| Sno   | Sname | Ssex | Sage | Sdept |
|-------|-------|------|------|-------|
| 95001 | 李勇    | 男    | 20   | CS    |
| 95002 | 刘晨    | 女    | 19   | CS    |
| 95003 | 王敏    | 女    | 18   | MA    |
| 95004 | 张立    | 男    | 19   | IS    |

**SC**

| Sno   | Cno | Grade |
|-------|-----|-------|
| 95001 | 1   | 92    |
| 95001 | 2   | 85    |
| 95001 | 3   | 88    |
| 95002 | 2   | 90    |
| 95002 | 3   | 80    |

连接结果:

| Student<br>.Sno | Sname | Ssex | Sage | Sdept | SC.Sno | Cno | Grade |
|-----------------|-------|------|------|-------|--------|-----|-------|
| 95001           | 李勇    | 男    | 20   | CS    | 95001  | 1   | 92    |
| 95001           | 李勇    | 男    | 20   | CS    | 95001  | 2   | 85    |
| 95001           | 李勇    | 男    | 20   | CS    | 95001  | 3   | 88    |
| 95002           | 刘晨    | 女    | 19   | CS    | 95002  | 2   | 90    |
| 95002           | 刘晨    | 女    | 19   | CS    | 95002  | 3   | 80    |

- 等值连接的一种特殊情况，把目标列中重复属性去掉

```
SELECT Student.Sno, Sname, Ssex, Sage, Sdept, Cno,  
Grade FROM Student, SC  
WHERE Student.Sno = SC.Sno;
```

连接结果：

| Student<br>.Sno | Sname | Ssex | Sage | Sdept | Cno | Grade |
|-----------------|-------|------|------|-------|-----|-------|
| 95001           | 李勇    | 男    | 20   | CS    | 1   | 92    |
| 95001           | 李勇    | 男    | 20   | CS    | 2   | 85    |
| 95001           | 李勇    | 男    | 20   | CS    | 3   | 88    |
| 95002           | 刘晨    | 女    | 19   | CS    | 2   | 90    |
| 95002           | 刘晨    | 女    | 19   | CS    | 3   | 80    |

## □非等值连接查询

**[<表名1>.]<列名1> <比较运算符> [<表名2>.]<列名2>**

- 比较运算符：>、<、>=、<=、!=

## □自身连接

- 一个表与其自己进行连接，称为表的自身连接
- 需要给表起别名以示区别
- 由于所有属性名都是同名属性，因此必须使用别名前缀

# (3) 自身连接

**[例]** 查询每门课的间接先修课，即先修课的先修课

```
SELECT FIRST.Cno, SECOND.Cpno  
FROM Course FIRST, Course SECOND  
WHERE FIRST.Cpno = SECOND.Cno;
```

**FIRST表 (Course表)**

| Cno | Cname    | Cpno | Ccredit |
|-----|----------|------|---------|
| 1   | 数据库      | 5    | 4       |
| 2   | 数学       |      | 2       |
| 3   | 信息系统     | 1    | 4       |
| 4   | 操作系统     | 6    | 3       |
| 5   | 数据结构     | 7    | 4       |
| 6   | 数据处理     |      | 2       |
| 7   | PASCAL语言 | 6    | 4       |

**SECOND表 (Course表)**

| Cno | Cname    | Cpno | Ccredit |
|-----|----------|------|---------|
| 1   | 数据库      | 5    | 4       |
| 2   | 数学       |      | 2       |
| 3   | 信息系统     | 1    | 4       |
| 4   | 操作系统     | 6    | 3       |
| 5   | 数据结构     | 7    | 4       |
| 6   | 数据处理     |      | 2       |
| 7   | PASCAL语言 | 6    | 4       |



## (3) 自身连接

**[例]** 查询每门课的间接先修课，即先修课的先修课

```
SELECT FIRST.Cno, SECOND.Cpno  
FROM Course FIRST, Course SECOND  
WHERE FIRST.Cpno = SECOND.Cno;
```

连接结果：

| FIRST.Cno | FIRST.Cpno | SECOND.Cno | SECOND.Cpno |
|-----------|------------|------------|-------------|
| 1数据库      | 5          | 5          | 7           |
| 3信息系统     | 1          | 1          | 5           |
| 4操作系统     | 6          | 6          |             |
| 5数据结构     | 7          | 7          | 6           |
| 7PASCAL语言 | 6          | 6          |             |

## (4) 外连接 (Outer Join)

### □ 外连接与普通连接的区别

- 普通连接操作只输出满足连接条件的元组
- 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出

### □ 左外连接

- 列出左边关系中所有的元组

### □ 右外连接

- 列出右边关系中所有的元组

## (4) 外连接 (Outer Join)

□[例] 查询每个学生及其选修课程的情况包括没有选修课程的学生

**SELECT Student.Sno, Sname, Ssex, Sage, Sdept, Cno, Grade FROM Student LEFT OUTER JOIN SC ON (Student.Sno = SC.Sno);**

| Student.Sno | Sname | Ssex | Sage | Sdept | Cno | Grade |
|-------------|-------|------|------|-------|-----|-------|
| 95001       | 李勇    | 男    | 20   | CS    | 1   | 92    |
| 95001       | 李勇    | 男    | 20   | CS    | 2   | 85    |
| 95001       | 李勇    | 男    | 20   | CS    | 3   | 88    |
| 95002       | 刘晨    | 女    | 19   | CS    | 2   | 90    |
| 95002       | 刘晨    | 女    | 19   | CS    | 3   | 80    |
| 95003       | 王敏    | 女    | 18   | MA    |     |       |
| 95004       | 张立    | 男    | 19   | IS    |     |       |

## (5) 复合条件连接

□ WHERE子句中含多个连接条件时，称为复合条件连接

- 复合条件连接可以看作（普通）连接后得到的关系（表）又进行一次选择运算

□ **[例]** 查询选修2号课程且成绩在90分以上的所有学生的学号、姓名

**SELECT Student.Sno, Sname**

**FROM Student, SC**

**WHERE Student.Sno = SC.Sno AND /\* 连接谓词\*/**

**SC.Cno='2' AND /\* 其他限定条件 \*/**

**SC.Grade>90; /\* 其他限定条件 \*/**

## (5) 复合条件连接

- 多表连接：连接操作涉及到两个以上的表的连接
- **[例]** 查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno, Sname, Cname, Grade  
FROM Student, SC, Course  
WHERE Student.Sno = SC.Sno  
AND SC.Cno = Course.Cno;
```

结果：

| Student.Sno | Sname | Cname | Grade |
|-------------|-------|-------|-------|
| 95001       | 李勇    | 数据库   | 92    |
| 95001       | 李勇    | 数学    | 85    |
| 95001       | 李勇    | 信息系统  | 88    |
| 95002       | 刘晨    | 数学    | 90    |
| 95002       | 刘晨    | 信息系统  | 80    |

## 4.4.1 数据查询

□1. 单表查询

□2. 连接查询

□3. 嵌套查询

- (1) 嵌套查询概述
- (2) 嵌套查询分类及求解方法
- (3) 引出子查询的谓词
  - ◆ 带有IN谓词的子查询
  - ◆ 带有比较运算符的子查询
  - ◆ 带有ANY或ALL谓词的子查询
  - ◆ 带有EXISTS谓词的子查询

□4. 集合查询

# (1) 嵌套查询概述

- 一个SELECT-FROM-WHERE语句称为一个查询块
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询

**SELECT Sname**

**FROM Student**      **-- 外层查询/父查询**

**WHERE Sno IN**

**(SELECT Sno FROM SC**      **--内层查询/子查询**  
**WHERE Cno='2');**

- 子查询的限制：不能使用ORDER BY子句
- 层层嵌套方式反映了SQL语言的结构化
- 有些嵌套查询可以用连接运算替代
- 嵌套查询的实现一般是从里到外，即先进行子查询，再把其结果用于父查询作为条件

## (2) 嵌套查询分类及求解方法

### □ 不相关子查询

- 子查询的查询条件不依赖于父查询
- 是由里向外逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件

### □ 相关子查询

- 子查询的查询条件依赖于父查询
- 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表
- 然后再取外层表的下一个元组
- 重复这一过程，直至外层表全部检查完为止



## (3) 引出子查询的谓词

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有ANY或ALL谓词的子查询
- 带有EXISTS谓词的子查询

# 带有IN谓词的子查询

**[例]** 查询与“刘晨”在同一个系学习的学生

□ 此查询要求可以分步来完成

① 确定“刘晨”所在系名

```
SELECT Sdept
```

```
FROM Student
```

```
WHERE Sname='刘晨';
```

结果为:

|       |
|-------|
| Sdept |
| CS    |

② 查找所有在CS系学习的学生

```
SELECT Sno, Sname, Sdept
```

```
FROM Student
```

```
WHERE Sdept='CS';
```

| Sno   | Sname | Sdept |
|-------|-------|-------|
| 95001 | 李勇    | CS    |
| 95002 | 刘晨    | CS    |

# 带有IN谓词的子查询

## □构造嵌套查询

- 将第一步查询嵌入到第二步查询的条件中

**SELECT Sno, Sname, Sdept**

**FROM Student**

**WHERE Sdept IN**

**(SELECT Sdept**

**FROM Student**

**WHERE Sname='刘晨');**

- 此查询为不相关子查询。DBMS求解该查询时也是分步去做的

# 带有IN谓词的子查询



- 父查询和子查询中的表均可以定义别名

```
SELECT Sno, Sname, Sdept
```

```
FROM Student S1
```

```
WHERE S1.Sdept IN
```

```
(SELECT Sdept
```

```
FROM Student S2
```

```
WHERE S2.Sname='刘晨');
```

- 用自身连接完成本查询要求

```
SELECT S1.Sno, S1.Sname, S1.Sdept
```

```
FROM Student S1, Student S2
```

```
WHERE S1.Sdept=S2.Sdept AND
```

```
S2.Sname='刘晨';
```

# 带有IN谓词的子查询

□[例] 查询选修了课程名为“信息系统”的学生学号和姓名

```
SELECT Sno, Sname  
FROM Student  
WHERE Sno IN
```

```
(SELECT Sno  
FROM SC
```

```
WHERE Cno IN
```

```
(SELECT Cno  
FROM Course
```

```
WHERE Cname='信息系统'));
```

③ 最后在Student关系中  
取出Sno和Sname

② 然后在SC关系找出选修了  
3号课程的学生学号

① 首先在Course关系找出“信  
息系统”的课程号，结果为3

# 带有IN谓词的子查询



## □使用连接查询

```
SELECT Student.Sno, Student.Sname  
FROM Student, SC, Course  
WHERE Student.Sno=SC.Sno  
AND SC.Cno=Course.Cno  
AND Course.Cname=‘信息系统’;
```

# 带有比较运算符的子查询

- 当能确切知道内层查询返回单值时，可用比较运算符 ( $>$ ,  $<$ ,  $=$ ,  $>=$ ,  $<=$ ,  $\neq$  或  $<>$ ) 与 ANY 或 ALL 谓词配合使用
- 例：假设一个学生只可能在一个系学习，并且必须属于一个系，则在左例可以用  $=$  代替 IN：

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN
  (SELECT Sdept
   FROM Student
   WHERE Sname='刘晨');
```

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept =
  (SELECT Sdept
   FROM Student
   WHERE Sname='刘晨');
```

# 带有比较运算符的子查询

□ 子查询一定要跟在比较符之后

□ 错误的例子：

```
SELECT Sno, Sname, Sdept
```

```
FROM Student
```

```
WHERE (SELECT Sdept
```

```
FROM Student
```

```
WHERE Sname='刘晨')
```

```
= Sdept;
```



# 带有ANY或ALL谓词的子查询



□谓词语义：ANY 任意一个值；ALL 所有值

□需要配合使用比较运算符

|              |                        |
|--------------|------------------------|
| > ANY        | 大于子查询结果中的某个值           |
| > ALL        | 大于子查询结果中的所有值           |
| < ANY        | 小于子查询结果中的某个值           |
| < ALL        | 小于子查询结果中的所有值           |
| >= ANY       | 大于等于子查询结果中的某个值         |
| >= ALL       | 大于等于子查询结果中的所有值         |
| <= ANY       | 小于等于子查询结果中的某个值         |
| <= ALL       | 小于等于子查询结果中的所有值         |
| = ANY        | 等于子查询结果中的某个值           |
| = ALL        | 等于子查询结果中的所有值（通常没有实际意义） |
| != (或<>) ANY | 不等于子查询结果中的某个值          |
| != (或<>) ALL | 不等于子查询结果中的任何一个值        |



# 带有ANY或ALL谓词的子查询



□[例] 查询其他系中比CS系任意一个（其中某一个）学生年龄小的学生姓名和年龄

```
SELECT Sname, Sage FROM Student
```

```
WHERE Sage < ANY (SELECT Sage FROM  
Student WHERE Sdept='CS')
```

```
AND Sdept <> 'CS'; /* 注意这是父查询块中的条件 */
```

## □执行过程

- DBMS执行此查询时，首先处理子查询，找出CS系中所有学生的年龄，构成一个集合 (20, 19)
- 处理父查询，找所有不是CS系且年龄小于19或20的学生

# 带有ANY或ALL谓词的子查询



□ ANY和ALL谓词有时可以用聚集函数实现

- ANY与ALL与聚集函数的对应关系

|     |    |         |       |        |       |        |
|-----|----|---------|-------|--------|-------|--------|
|     | =  | <> 或 != | <     | <=     | >     | >=     |
| ANY | IN | --      | < MAX | <= MAX | > MIN | >= MIN |
| ALL | -- | NOT IN  | < MIN | <= MIN | > MAX | >= MAX |

- 用聚集函数实现子查询通常比直接用ANY或ALL查询效率要高，因为前者通常能够减少比较次数

# 带有ANY或ALL谓词的子查询



□[例] 用聚集函数实现查询其他系中比CS系任意一个（其中某一个）学生年龄小的学生姓名和年龄

```
SELECT Sname, Sage
```

```
FROM Student
```

```
WHERE Sage <
```

```
(SELECT MAX(Sage)
```

```
FROM Student WHERE Sdept='CS')
```

```
AND Sdept <> 'CS';
```

# 带有ANY或ALL谓词的子查询



□[例] 查询其他系中比CS系所有学生年龄都小的学生姓名及年龄

● 方法一：用ALL谓词

```
SELECT Sname, Sage
```

```
FROM Student
```

```
WHERE Sage < ALL
```

```
(SELECT Sage
```

```
FROM Student
```

```
WHERE Sdept='CS')
```

```
AND Sdept <> 'CS';
```

● 方法二：用聚集函数

```
SELECT Sname, Sage
```

```
FROM Student
```

```
WHERE Sage <
```

```
(SELECT MIN(Sage)
```

```
FROM Student
```

```
WHERE Sdept='CS')
```

```
AND Sdept <> 'CS';
```



# 带有EXISTS谓词的子查询

1. EXISTS谓词
2. NOT EXISTS谓词
3. 不同形式的查询间的替换
4. 用EXISTS/NOT EXISTS实现全称量词
5. 用EXISTS/NOT EXISTS实现逻辑蕴涵

# 带有EXISTS谓词的子查询

## 1. EXISTS谓词 (存在量词 $\exists$ )

- 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”
  - ◆ 若内层查询结果非空，则返回真值
  - ◆ 若内层查询结果为空，则返回假值
- 由EXISTS引出的子查询，其目标列表达式通常用\*，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义

## 2. NOT EXISTS谓词

# 带有EXISTS谓词的子查询

**[例]** 查询所有选修了1号课程的学生姓名

□思路分析：本查询涉及Student和SC关系

- 在Student中依次取每个元组的Sno值，用此值去检查SC关系
- 若SC中存在这样的元组，其Sno值等于此Student.Sno值，且其Cno='1'，则取此Student.Sname送入结果关系

## 用嵌套查询

```
SELECT Sname
FROM Student
WHERE EXISTS
  (SELECT *
   FROM SC /*相关子查询*/
   WHERE Sno=Student.Sno
    AND Cno='1');
```

## 用连接运算

```
SELECT Sname
FROM Student, SC
WHERE
  Student.Sno=SC.Sno
  AND SC.Cno='1';
```



# 带有EXISTS谓词的子查询

## □相关子查询

- 子查询的查询条件依赖于父查询
- 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表
- 然后再取外层表的下一个元组
- 重复这一过程，直至外层表全部检查完为止

## □[例] 查询没有选修1号课程的学生姓名

```
SELECT Sname  
FROM Student  
WHERE NOT EXISTS  
  (SELECT * FROM SC  
   WHERE Sno=Student.Sno AND Cno='1');
```

# 带有EXISTS谓词的子查询

## 3. 不同形式的查询间的替换

- **一些**带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
- **所有**带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换
- 带有EXISTS谓词的相关子查询只关心内层查询是否有返回值，不需要查具体值，效率不一定低于不相关子查询

□例：查询与“刘晨”在同一个系学习的学生。可以用带EXISTS谓词的子查询替换：

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN
  (SELECT Sdept
   FROM Student
   WHERE Sname='刘晨');
```

```
SELECT Sno, Sname, Sdept
FROM Student S1
WHERE EXISTS
  (SELECT * FROM Student S2
   WHERE S2.Sdept=S1.Sdept
   AND S2.Sname='刘晨');
```

# 带有EXISTS谓词的子查询

## 4. 用EXISTS/NOT EXISTS实现全称量词

- SQL语言中没有全称量词  $\forall$  (For all)
- 可以把带有全称量词的谓词转换为等价的带有存在量词的谓词:

$$(\forall x)P \equiv \neg(\exists x(\neg P))$$

# 带有EXISTS谓词的子查询

□[例4-27] 查询选修了全部课程的学生姓名

```
SELECT Sname  
FROM Student  
WHERE NOT EXISTS  
  (SELECT *  
   FROM Course  
   WHERE NOT EXISTS  
     (SELECT *  
      FROM SC  
      WHERE Sno=Student.Sno  
      AND Cno=Course.Cno));
```

这样的课是不存在的

这门课他没选

$(\forall x)P \equiv \neg(\exists x(\neg P))$   
没有一门课程是他不选的

# 带有EXISTS谓词的子查询

## 5. 用EXISTS/NOT EXISTS实现逻辑蕴涵

- SQL语言中没有蕴涵 (Implication) 逻辑运算
- 可以利用谓词演算将逻辑蕴涵谓词等价转换为：

$$p \rightarrow q \equiv \neg p \vee q$$

# 带有EXISTS谓词的子查询

**[例]** 查询至少选修了学生95001选修的全部课程的学生号码

□ 解题思路：用逻辑蕴涵表达

- 查询学号为x的学生，对所有的课程y，只要95001学生选修了课程y，则x也选修了y

□ 形式化表示：

- 用p表示谓词 “学生95001选修了课程y”
- 用q表示谓词 “学生x选修了课程y”
- 则上述查询为： $(\forall y) p \rightarrow q$

□ 等价变换：

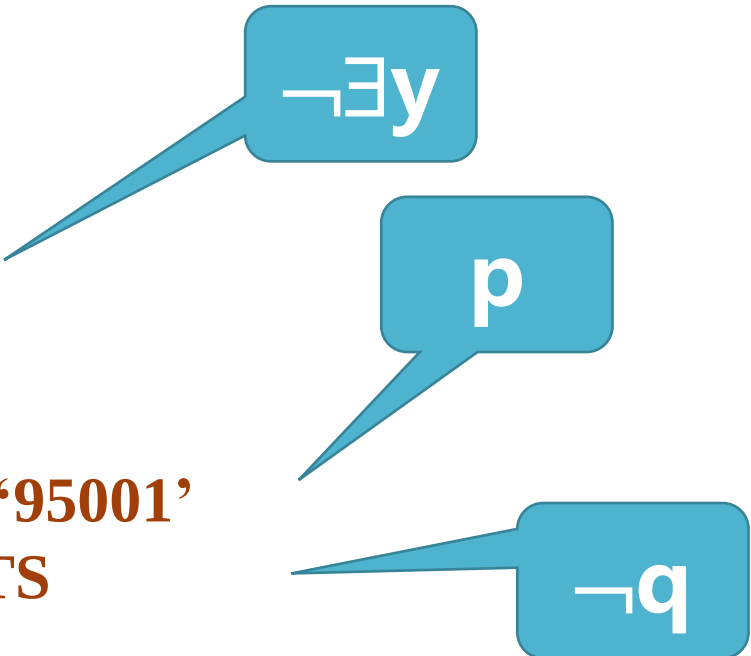
- $(\forall y)p \rightarrow q \equiv \neg(\exists y(\neg(p \rightarrow q))) \equiv \neg(\exists y(\neg(\neg p \vee q))) \equiv \neg\exists y(p \wedge \neg q)$
- 变换后语义：不存在这样的课程y，学生95001选修了y，而学生x没有选

# 带有EXISTS谓词的子查询

- 用NOT EXISTS谓词表示：不存在这样的课程y，学生95001选修了y，而学生x没有选

$$\neg \exists y (p \wedge \neg q)$$

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS
  (SELECT *
   FROM SC SCY
   WHERE SCY.Sno='95001'
    AND NOT EXISTS
      (SELECT *
       FROM SC SCZ
       WHERE SCZ.Sno=SCX.Sno AND
        SCZ.Cno=SCY.Cno));
```





## 4.4.1 数据查询

- 1. 单表查询
- 2. 连接查询
- 3. 嵌套查询
- 4. 集合查询



## 4. 集合查询

### □ 标准SQL直接支持的集合操作种类

- 并操作 (UNION)

### □ 一般商用数据库支持的集合操作种类

- 并 (UNION)、交 (INTERSECT)、差 (MINUS/EXCEPT)

### □ 原则

- 属性个数必须一致
- 对应的类型必须一致
- 属性名无关
- 最终结果集采用第一个结果的属性名
- 缺省为自动去除重复元组，除非显式说明ALL
- ORDER BY放在整个语句的最后

# (1) 并操作

## □形式

<查询块>

UNION [ALL]

<查询块>;

- 参加UNION操作的个结果集列数必须相同，而且对应项的数据类型必须兼容
- 使用UNION合并多个结果集时，系统会自动去掉重复元组
- 如果需要保留重复元组，使用UNION ALL操作符

# (1) 并操作

□[例] 查询计算机科学系的学生或者年龄不大于19岁的学生，并按年龄倒排序

● 方法一：

```
SELECT *  
    FROM Student  
    WHERE Sdept='CS'  
UNION  
SELECT *  
    FROM Student  
    WHERE Sage<=19  
ORDER BY Sage DESC;
```

● 方法二：

```
SELECT DISTINCT *  
    FROM Student  
    WHERE Sdept='CS' OR  
          Sage<=19  
ORDER BY Sage DESC;
```

## (2) 交操作

□[例] 查询计算机系的学生并且年龄不大于19岁的学生，并按年龄倒排序

- 标准SQL中没有提供集合交操作，但可用其他方法间接实现。交操作可以使用AND操作符或子查询实现
- 本例实际上就是查询计算机科学系中年龄不大于19岁的学生

```
SELECT *  
  FROM Student  
 WHERE Sdept='CS'  
INTERSEC  
SELECT *  
  FROM Student  
 WHERE Sage<=19  
ORDER BY Sage DESC;
```

```
SELECT *  
  FROM Student  
 WHERE Sdept='CS'  
        AND Sage<=19  
ORDER BY Sage DESC;
```

## (2) 交操作

□[例] 查询选修课程1的学生集合与选修课程2的学生集合的交集

- 本例实际上是查询既选修了课程1又选修了课程2的学生

```
SELECT Sno
FROM SC
WHERE Cno='1' AND Sno IN
    (SELECT Sno
     FROM SC
     WHERE Cno='2');
```

## (3) 差操作



□[例] 查询选修课程1但没有选修课程2的学生姓名

```
SELECT Sname
FROM Student
WHERE Sno IN
  ((SELECT Sno
    FROM SC
    WHERE Cno='1')
  EXCEPT
  (SELECT Sno
    FROM SC
    WHERE Cno='2'));
```

## (3) 差操作

□ 标准SQL中没有提供集合差操作，但可用其他方法间接实现

- 集合差操作可以把差操作转化为适当的普通查询

□ **[例]** 查询计算机科学系的学生与年龄不大于19岁的学生的差集

- 本例实际上是查询计算机科学系中年龄大于19岁的学生

**SELECT \***

**FROM Student**

**WHERE Sdept='CS' AND Sage>19;**

## (4) 对集合操作结果的排序

- ❑ ORDER BY子句只能用于对最终查询结果排序，不能对中间结果排序
- ❑ 任何情况下，ORDER BY子句只能出现在最后
  - 对集合操作结果排序时，ORDER BY子句中用数字指定排序属性

### [例] 错误写法

```
SELECT * FROM Student
  WHERE Sdept='CS'
  ORDER BY Sno
UNION
SELECT * FROM Student
  WHERE Sage<=19
  ORDER BY Sno;
```

### 正确写法

```
SELECT * FROM Student
  WHERE Sdept='CS'
UNION
SELECT * FROM Student
  WHERE Sage<=19
  ORDER BY 1;
```



# SELECT语句的一般格式

**SELECT** [ ALL|DISTINCT ]

<目标列表达式> [别名] [, <目标列表达式> [别名]] ...

**FROM** <表名或视图名> [别名] [, <表名或视图名> [别名]] ...

[ **WHERE** <条件表达式> ]

[ **GROUP BY** <列名1> [ **HAVING** <条件表达式> ] ]

[ **ORDER BY** <列名2> [ ASC|DESC ] ];

## □实验1：关系数据库系统环境和数据库建立

### □实验目的

- 理解和掌握数据库DDL语言

### □实验内容

- 详见乐学课程

### □提交方式

- 打包提交：实验报告、源文件（.sql文件）
- 压缩文件命名方式：学号-姓名-实验X

### □助教：冯齐华（i北理课程群）